Plan complet des scénarios de test - Ecopath+

Testeurs :
AHMED MUSHTAQ Léo,
APEDOH SENAM KOSSI AMENYO Jules,
CLOUET Louis,
LETEURTRE Martin,
SICHI Axel,
SMETS Yoann

Projet : Ecopath+ (UPSSITECH 2025 RED)

2 février 2026

ID Feature : UPSS-F6-UC2 / UPSS-F6-UC3
Domaine : Matériel / Physique
Titre : Validation du fonctionnement et calibration des capteurs (Ultrasons/Caméras)

1. Description du cas de test

Ce cas de test vise à valider l'intégration physique et logicielle des capteurs sur le robot. Il s'agit de vérifier que les données remontent correctement dans l'environnement ROS2, que la détection d'obstacle est effective et que les mesures sont calibrées par rapport à la réalité physique.

2. Préconditions

- Le robot est assemblé et les capteurs (ultrasons/caméras) sont branchés physiquement.
- L'environnement de développement (ROS2) est lancé et les drivers des capteurs sont actifs.
- Un mètre ruban est disponible pour vérifier les distances réelles.
- Une zone dégagée d'au moins 3 mètres est disponible face au robot.

3. Étapes de test

Étape 1 : Vérification de la remontée de données (Topic ROS)

Given Que le robot est sous tension et que les nœuds ROS des capteurs sont lancés.

When L'utilisateur exécute la commande d'écoute du topic (ex : `ros2 topic echo /sensors/ultrasound`).

Then Des valeurs numériques doivent s'afficher en continu dans le terminal sans erreur de timeout.

Étape 2 : Test de calibration (Précision de la mesure)

Given Que les capteurs envoient des données.

When Un obstacle plat est positionné précisément à **50 cm** puis à **100 cm** du capteur.

Then La valeur retournée par le capteur doit correspondre à la distance réelle avec une marge d'erreur inférieure à 5% (ex : entre 47.5cm et 52.5cm pour la mesure à 50cm).

Étape 3 : Détection en conditions réelles (Dynamique)

Given Que le robot est en mouvement (ou qu'un obstacle se déplace vers lui).

When L'obstacle entre dans le champ de détection (< 30 cm).

Then Le système doit détecter la présence de l'obstacle immédiatement et la donnée doit se mettre à jour en temps réel (latence < 200ms).

4. Données de test

- **Distances de référence :** 20 cm, 50 cm, 100 cm, 200 cm.
- **Types d'obstacles :** Mur blanc (réflexion standard), Carton sombre (faible réflexion), Objet cylindrique (pour tester la dispersion).
- **Environnement :** Pièce éclairée, sol plat.

5. Résultat attendu

- ✓ Les capteurs doivent être correctement reconnus par le système (topics ROS actifs).
- ✓ Les distances mesurées doivent être cohérentes avec la réalité physique (Calibration validée).
- ✓ Aucun faux positif (détection fantôme) ne doit apparaître lorsque la zone est vide.
- ✓ Le bruit de mesure doit rester stable (pas de saut de valeur aberrant type 0m ou 50m instantané).

6. Résultat réel

[À compléter manuellement lors de l'exécution du test sur le robot physique.]

7. Statut

ÉTAPE	DEV	INT	VAL	PROD
E1 : Remontée de données	☒	☐	☐	☐
E2 : Calibration Précision	☐	☐	☐	☐
E3 : Détection dynamique	☐	☐	☐	☐

ID Feature : UPSS-F5-UC1 / UPSS-F8-UC2
Domaine : Simulation (Digital Twin)
Titre : Validation de l'environnement simulé et des capteurs virtuels

1. Description du cas de test

Ce scénario vise à valider le bon fonctionnement du jumeau numérique du robot dans le simulateur Gazebo. Il s'assure que le modèle URDF est correctement chargé, que la physique (gravité, collision) est active et que les capteurs simulés (Lidar/Caméra) publient des données exploitables sur les topics ROS2.

2. Préconditions

- L'environnement ROS2 (Humble/Jazzy) et Gazebo sont installés sur la VM ou le Mac.
- Le package de description du robot (‘robot$_d$*escription*‘)*estcompil*.
- Le fichier ‘launch‘ de la simulation est configuré.

3. Étapes de test

Étape 1 : Chargement de l'environnement et Spawn

Given Que le workspace est sourcé.

When L'utilisateur lance la commande de simulation (ex : `ros2 launch my_bot_gazebo world.launch.py`).

Then Gazebo doit s'ouvrir sans erreur critique. Le robot doit apparaître au point de spawn (0,0,0) sans s'enfoncer dans le sol ni trembler (stabilité physique).

Étape 2 : Vérification de la cinématique (Téléopération)

Given Que la simulation est active.

When Un nœud de téléopération est lancé (`teleop_twist_keyboard`) et une commande de vitesse linéaire ($v_x > 0$) est envoyée.

Then Le robot virtuel doit avancer visuellement dans Gazebo et les données d'odométrie (`/odom`) doivent s'incrémenter de manière cohérente.

Étape 3 : Validation des capteurs virtuels (Lidar/Caméra)

Given Que le robot est positionné face à un obstacle virtuel (ex : cube ou mur).

When On visualise les topics via Rviz2 ou `ros2 topic echo`.

Then
- Le topic `/scan` doit renvoyer des points correspondant à la distance de l'obstacle.
- Le topic `/camera/image_raw` doit afficher la vue simulée de l'obstacle.

4. Données de test

- **Monde Gazebo :** `empty_world` pour le test physique, `maze_world` pour le test capteurs.
- **Commande de vitesse :** $v_{lineaire} = 0.2$ m/s, $v_{angulaire} = 0.5$ rad/s.
- **Obstacle :** Cube de 1m x 1m placé à $x = 2.0m$ devant le robot.

5. Résultat attendu

- ✓ Le modèle du robot (URDF/Xacro) s'affiche avec les bonnes textures et dimensions.
- ✓ Le robot interagit avec la physique (il percute les murs, ne passe pas au travers).
- ✓ Les capteurs simulés publient à la fréquence attendue (ex : Lidar à 10Hz, Caméra à 30Hz).
- ✓ La transformation TF (`map -> odom -> base_link`) est publiée et continue.

6. Résultat réel

[À compléter après exécution : noter si latence observée dans Gazebo ou erreurs de TF.]

7. Statut

ÉTAPE	DEV	INT	VAL	PROD
E1 : Spawn & Physique	☐	☐	☐	☐
E2 : Cinématique	☐	☐	☐	☐
E3 : Capteurs Virtuels	☐	☐	☐	☐

ID Feature : UPSS-F9-UC3 / UPSS-F7-UC3
Domaine : Robustesse & Sécurité
Titre : Validation de la résilience système (Failsafe, Crash Process, Charge CPU)

1. Description du cas de test

Ce cahier de test a pour objectif de valider le comportement du robot en **conditions dégradées**. Il s'agit de s'assurer que le système réagit de manière sécuritaire (Fail-Safe) en cas de perte de communication ou de crash logiciel, et qu'il maintient ses performances de navigation même en cas de surcharge processeur (Stress Test).

2. Préconditions

- Le robot est déployé (physique ou simulation Gazebo) avec ROS2 Humble/Jazzy.
- L'accès SSH est configuré.
- Les outils de monitoring système sont installés (`htop`, `stress` ou `stress-ng`).
- La configuration du `ros2_control` ou du pilote moteur inclut un paramètre de timeout (watchdog).

3. Étapes de test

Scénario 1 : Test de Failsafe Réseau (Watchdog)

Objectif : Vérifier que le robot ne continue pas sa course si l'opérateur perd la connexion.

Given Le robot est téléopéré manuellement et avance à une vitesse constante ($v > 0$).

When La connexion réseau est coupée brutalement (désactivation du Wi-Fi contrôleur ou fermeture du terminal SSH) sans relâcher la commande d'avance.

Then Le robot doit s'arrêter de lui-même après un délai court (timeout < 1 seconde). Les moteurs doivent revenir à une vitesse nulle.

Scénario 2 : Résilience Processus (ROS2 Lifecycle)

Objectif : Vérifier la réaction du système à la perte d'un nœud critique (Lidar ou Caméra).

Given Le robot est en fonctionnement nominal, tous les capteurs publient des données.

When L'ingénieur tue le processus d'un capteur critique (ex : `kill -9 <PID_LIDAR>`).

Then
- Le système de navigation (Nav2) doit détecter la perte de données et passer en état d'erreur/arrêt.
- (Optionnel) Le gestionnaire de lancement (launch file avec `respawn='true'`) tente de relancer le nœud automatiquement.

Scénario 3 : Test de Charge (CPU Stress Test)

Objectif : Vérifier que la boucle de navigation reste prioritaire sur la charge CPU.

Given Le robot effectue une navigation autonome simple d'un point A à un point B.

When Une charge artificielle est lancée sur le CPU (ex : commande `stress -cpu 4` ou compilation en parallèle).

Then Le robot ne doit pas saccader de manière dangereuse. La fréquence du topic d'odométrie doit rester stable (variation tolérée $< 20\%$). Aucune collision ne doit survenir.

4. Données de test

- **Paramètre Timeout cmd_vel :** 0.5 secondes.
- **Nœud cible (Kill) :** Driver Lidar (`rplidar_node` ou `ld06`) ou Caméra.
- **Charge CPU :** Utilisation de `stress-ng` pour saturer 90%+ des cœurs CPU pendant 30 secondes.

5. Résultat attendu

✓ **Sécurité :** Arrêt immédiat du robot en cas de perte de signal (pas de "robot fou").

✓ **Intégrité :** Le système ne plante pas globalement si un seul nœud crash (isolation).

✓ **Stabilité :** La boucle de contrôle temps réel reste fonctionnelle sous charge (priorité processus).

6. Résultat réel

[À compléter lors de la session de crash-test.]

7. Statut

SCÉNARIO	DEV	INT	VAL	PROD
S1 : Network Watchdog	☐	☐	☐	☐
S2 : Crash Nœud Safety	☐	☐	☐	☐
S3 : Stress Test CPU	☐	☐	☐	☐

ID Feature : UPSS-F8-UC2 / UPSS-F8-UC4
Domaine : Algorithmique / SLAM
Titre : Validation de la génération de carte (Loop Closure, Précision, Drift)

1. Description du cas de test

Ce cahier de test vise à valider la qualité de la cartographie générée par le robot (via SLAM Toolbox ou Cartographer). Il s'agit de vérifier la cohérence géométrique de la carte lors des déplacements longs (fermeture de boucle), la précision des dimensions réelles (fidélité métrique) et la stabilité de l'estimation de pose lors des rotations rapides (résistance au drift).

2. Préconditions

- Le robot est opérationnel (Lidar et Odométrie publient sur ROS2).
- Le nœud de SLAM est lancé (ex : `ros2 launch slam_toolbox online_async_launch.py`).
- Rviz2 est ouvert avec le plugin *Map* et l'outil *Measure* activé.
- Environnement de test : Une zone avec une boucle possible (ex : couloir autour d'un bureau) et un mur droit mesurable.

3. Étapes de test

Scénario 1 : Test de Fermeture de Boucle (Loop Closure)

Objectif : Vérifier la capacité du SLAM à reconnaître un lieu déjà visité et corriger la dérive.

Given Le robot démarre d'un point A (0,0,0), la cartographie est active.

When L'opérateur pilote le robot pour effectuer une boucle complète (retour au point A après avoir parcouru $>$ 10m).

Then Lors du retour au point A, le logiciel doit détecter la correspondance ("Loop Closure Detected"). La carte doit se recaler proprement : les murs du début et de la fin doivent se superposer parfaitement sans effet de dédoublement.

Scénario 2 : Test de Fidélité Métrique (Précision)

Objectif : Valider l'échelle de la carte par rapport à la réalité.

Given Un mur physique rectiligne de longueur connue (mesuré au mètre ruban, ex : $L_{reel} = 2.00m$).

When Le robot cartographie ce mur. L'utilisateur utilise l'outil "Measure" de Rviz2 pour mesurer la distance entre les pixels de début et de fin du mur sur la carte (L_{map}).

Then La mesure virtuelle doit correspondre à la réalité avec une erreur inférieure à 5%. (Calcul : $|L_{reel} - L_{map}|/L_{reel} < 0.05$).

Scénario 3 : Test de Stabilité Rotationnelle (Drift)

Objectif : Vérifier que l'odométrie et le Lidar ne décrochent pas en rotation rapide.

Given Le robot est statique au centre d'une pièce cartographiée. Les murs sont nets sur Rviz.

When L'opérateur effectue une rotation sur place de 360° (vitesse angulaire ~ 0.5 rad/s).

Then Durant la rotation, les murs affichés sur la carte ne doivent pas bouger ni devenir flous ("ghosting"). À la fin du tour, le robot doit retrouver son orientation initiale sur Rviz sans décalage angulaire majeur.

4. Données de test

- **Longueur Mur Référence :** 200 cm (ou autre valeur mesurée).
- **Vitesse de rotation :** Testé à 0.2 rad/s (lent) puis 0.5 rad/s (rapide).
- **Paramètres SLAM :** Mode `online_async`, résolution de grille 0.05m.

5. Résultat attendu

✓ **Cohérence :** Une seule carte unifiée, pas de couloirs qui se croisent ou se dédoublent.

✓ **Précision :** Un mur de 2m mesure entre 1.90m et 2.10m sur la carte.

✓ **Stabilité :** Pas d'apparition d'artefacts (points parasites) lors des rotations.

6. Résultat réel

[À compléter manuellement lors des essais. Noter ici les valeurs mesurées dans Rviz.]

7. Statut

SCÉNARIO	DEV	INT	VAL	PROD
S1 : Loop Closure	☐	☐	☐	☐
S2 : Fidélité Métrique	☐	☐	☐	☐
S3 : Stabilité Rotation	☐	☐	☐	☐